

DL050 RNN 与 BPTT 详细答案

本答案与 lab.md 的 A 至 H 任务一一对应。数值均由本包实际运行产生，表中四舍五入不应用于重新训练。结论适用于预定的合成任务、模型与预算；没有把检查通过当作任意输入正确性证明，也没有把 toy 高准确率当自然语言泛化证据。

1 任务 A 数据 状态 参数与归约

A 的长度是 3，B 的长度是 2；两行状态 mask 和逐位置损失 mask 都是 $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$ 。有效位置数为 5。补齐输入与标签是存储占位，不是新的观测。

参数是 U、W、b、v、c，在所有样本和时刻重复使用。状态 h 随具体输入变化，每个样本有各自的 h_0 。数据是输入 x、标签 y、实际长度，正常监督学习中不被优化器更新。输入梯度用于诊断或传回上游嵌入层，与把原始数据设为待学习参数不同。

逐有效位置平均时每个位置权重是 $1/5$ 。若每条序列先平均再对两条序列平均，A 的每个位置权重是 $(1/2)(1/3)=1/6$ ，B 的每个位置权重是 $(1/2)(1/2)=1/4$ 。这两个目标不同，因此梯度一般不同。要从每条序列自己的均值梯度恢复 token-mean，应乘 $L_i/\sum L_i$ 再相加。

D=2、H=3、N=4、T=5 时，X 为 $\begin{bmatrix} 4 & 5 & 2 \end{bmatrix}$ ，U 为 $\begin{bmatrix} 3 & 2 \end{bmatrix}$ ，W 为 $\begin{bmatrix} 3 & 3 \end{bmatrix}$ ，b 和 v 为 $\begin{bmatrix} 3 \end{bmatrix}$ ，c 为零维，状态为 $\begin{bmatrix} 4 & 5 & 3 \end{bmatrix}$ ，logit 为 $\begin{bmatrix} 4 & 5 \end{bmatrix}$ 。总参数数为 $3 \times 2 + 3 \times 3 + 2 \times 3 + 1 = 22$ 。X[:,t] 为 $\begin{bmatrix} 4 & 2 \end{bmatrix}$ ，右乘 U.T 的 $\begin{bmatrix} 2 & 3 \end{bmatrix}$ 得到 $\begin{bmatrix} 4 & 3 \end{bmatrix}$ ，其每一行就是列向量 Ux 的转置。

若 $x=E[\text{token}]$ ，RNN 给出每个位置的 dx，再按 token ID 对 E 的行 scatter-add。同一 token 多次出现，贡献相加。不能对整数 ID 施加微小实数扰动并当作有效 token；查表的可训练对象是 E 的连续数值。

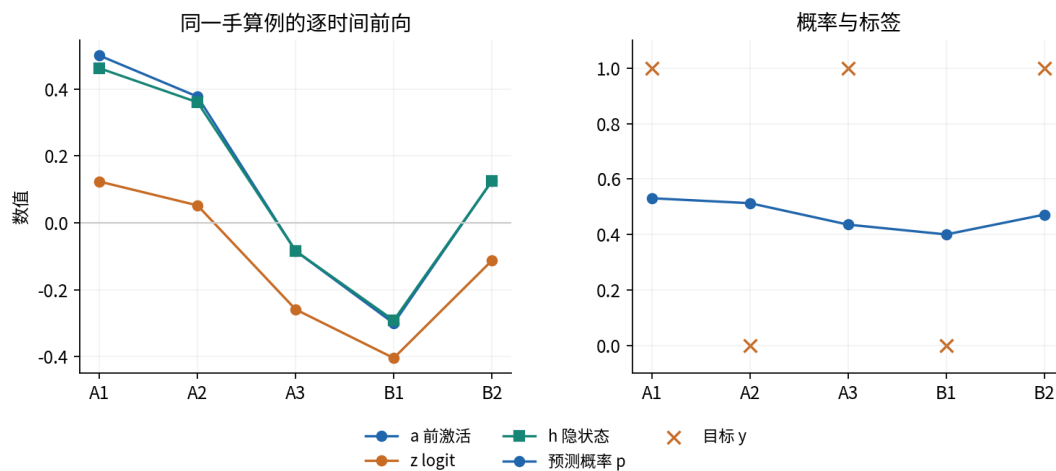
2 任务 B 完整数值链

2.1 前向账本

初始参数 $U=0.4$ 、 $W=0.6$ 、 $b=0.1$ 、 $v=0.7$ 、 $c=-0.2$ ，两条序列的 h_0 都为零。A1 的 $a=0.5$ 、 $h=0.462117$ ；A2 必须接 A1，得到 $a=0.377270$ ；A3 接 A2，得到 $a=-0.083799$ 。B1 则重新从零开始，得到 $a=-0.3$ 。

样本时刻	x	y	a	h	z	p	单位置损失
A1	1.000000	1	0.500000	0.462117	0.123482	0.530831	0.633311
A2	0.000000	0	0.377270	0.360335	0.052234	0.513056	0.719605
A3	-1.000000	1	-0.083799	-0.083604	-0.258523	0.435727	0.830740
B1	-1.000000	0	-0.300000	-0.291313	-0.403919	0.400371	0.511444
B2	0.500000	1	0.125212	0.124562	-0.112806	0.471828	0.751140

五个有效 BCE 的和为 3.446240439，平均 $L=0.689248088$ 。B3 采用的状态为 $h_{B2}=0.124562144$ ；未采用的候选不应被误当作状态。B3 没有损失，输入梯度为零。



前向数值可视化。这里的折线用于比较各位置数值，并非跨样本状态连接。

2.2 完整反向账本

每个输出先产生 $e=(p-y)/5$ 。对某个状态，直接输出梯度 ve 与未来状态梯度 $w\delta$ 下一步相加，再乘 $\tanh$ 导数。末尾未来项为零。

位置	$e = (p-y)/5$	直接 ve	未来 $w\delta$ 下一步	总 r	局部 $1-h^2$	δ
A1	-0.093834	-0.065684	0.012927	-0.052756	0.786448	-0.041490
A2	0.102611	0.071828	-0.047068	0.024760	0.870159	0.021545
A3	-0.112855	-0.078998	0.000000	-0.078998	0.993010	-0.078446
B1	0.080074	0.056052	-0.043678	0.012374	0.915137	0.011324
B2	-0.105634	-0.073944	0.000000	-0.073944	0.984484	-0.072797

A2 直接项为正，因为该位置目标为 0，降低当前 logit 有助降低损失；A3 目标为 1，其损失产生负方向的未来影响。两条路径作用于同一个 h_{A2} ，必须相加，因此部分抵消。没有即时损失的时刻仍可能从未来得到梯度。

A1 的 δ 分解如下：

到达 A1 的损失来源	对 a_{A1} 的梯度
$\ell_{A1/5}$	-0.051657
$\ell_{A2/5}$	0.029493
$\ell_{A3/5}$	-0.019326
三条路径之和	-0.041490

其中来自 A3 的路径穿过 A3、A2、A1 三个 $\tanh$ 导数以及两次 W 。写成链式积后再相加，与逆序递推的 $\delta_{A1} = -0.041490196$ 一致。用一条“总箭头”无法替代这些不同路径的求和解释。

2.3 共享参数贡献

位置	δx 对 U	$\delta h_{前}$ 对 W	δ 对 b	$e h$ 对 v	e 对 c
A1	-0.041490	-0.000000	-0.041490	-0.043362	-0.093834
A2	0.000000	0.009956	0.021545	0.036974	0.102611
A3	0.078446	-0.028267	-0.078446	0.009435	-0.112855
B1	-0.011324	0.000000	0.011324	-0.023327	0.080074
B2	-0.036398	0.021207	-0.072797	-0.013158	-0.105634
总和	-0.010766	0.002896	-0.159864	-0.033437	-0.129637

W 的局部梯度等于 δ 乘进入该位置的旧状态。A1、B1 的 h_0 都为 0，所以两个局部项为 0；A2 的 $x=0$ ，故其 U 局部项为 0，但 δ_{A2} 不是 0。这说明“某个参数没有局部贡献”与“这个时刻不参与反传”完全不同。

2.4 同步更新与新前向

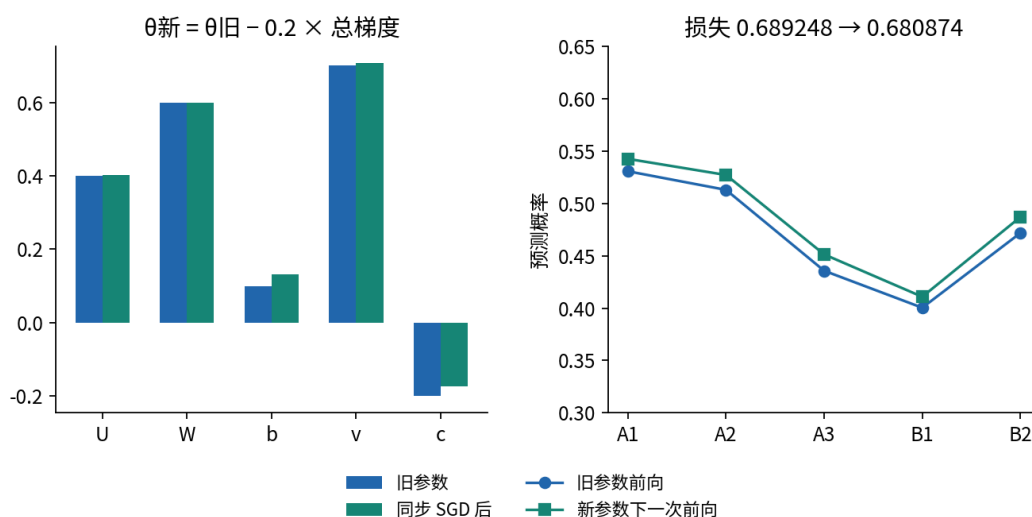
$\eta=0.2$ ，全部梯度使用同一组旧参数，更新如下。

参数	旧值	总梯度	学习率 × 梯度	新值
U	0.400000	-0.010766	-0.002153	0.402153
W	0.600000	0.002896	0.000579	0.599421
b	0.100000	-0.159864	-0.031973	0.131973
v	0.700000	-0.033437	-0.006687	0.706687
c	-0.200000	-0.129637	-0.025927	-0.174073

新参数重新前向：

位置	新 a	新 h	新 z	新 p	新损失
A1	0.534126	0.488529	0.171165	0.542687	0.611223
A2	0.424807	0.400972	0.109290	0.527295	0.749284
A3	-0.029829	-0.029820	-0.195146	0.451368	0.795473
B1	-0.270180	-0.263793	-0.360492	0.410841	0.529058
B2	0.174927	0.173164	-0.051700	0.487078	0.719331

新均值损失为 0.680873898，降低约 0.008374190。A2 从 0.719605278 上升至 0.749284282，B1 从 0.511444422 上升至 0.529058460；其他位置下降更多，因此总平均下降。优化共同参数时，不保证每个位置都同时改善。



同步更新和下一次前向。总损失下降不要求每个位置都下降。

3 任务 C 独立反传的完整结构

单个有效时刻的计算图为 $a=Ux+Wh$ 前+b、 $h=\tanh(a)$ 、 $z=v^T h+c$ 。设 e 为含归约因子的输出梯度， δ 为前激活梯度，则

$$\begin{aligned} r_t &= v e_t + W^T \delta_{t+1}, \quad \delta_t = r_t \odot (1 - h_t^2). \\ dU &= \sum \delta_t x_t^T, \quad dW = \sum \delta_t h_{t-1}^T, \quad db = \sum \delta_t. \\ dv &= \sum e_t h_t, \quad dc = \sum e_t, \quad dx_t = U^T \delta_t. \end{aligned}$$

求和覆盖全部样本与有效时间。参考实现实际保存所有候选和状态；补齐位置使用 mask 后的状态。严格反传是 $\delta = m r \odot (1 - \text{candidate}^2)$ ，上一步梯度为 $W^T \delta + (1 - m) r$ 。若省略恒等分支，一般冻结状态图会被错误求导。

实现顺序：前向缓存每步 a 、candidate 和 h ；算所有 z 、 e ；从末步倒序维护 dh_{next} ；形成 direct+future；计算 δ ；累加五个参数梯度；记录 dx ；最后更新 dh_{next} 。此过程中不修改参数。最后输出 dh_0 供诊断，固定初态不更新。

3.1 为什么需要矩阵测试

$H=D=1$ 时，转置是无效操作，沿错误轴求和也可能碰巧得到相同形状。 $N=3$ 、 $T=4$ 、 $D=2$ 、 $H=3$ 、长度 $[4,2,1]$ 能暴露： U 的转置错误、 W 的左右乘错误、bias 广播的错误轴、序列长度不同导致的分母错误，以及 padding 状态错误。

提供的检查同时对每个参数与每个输入做中心有限差分，包括 padding 输入。纸笔例最大绝对误差实测如下：

ϵ	全部参数的最大绝对误差
0.0001	1.955e-09
1e-05	2.080e-11
1e-06	5.411e-11

这只是选定点的有限差分结果。 ϵ 降低时不保证误差严格单调下降，因为截断误差与浮点抵消在竞争。测试设置绝对与相对容差，对接近零的梯度优先检查绝对误差。浮点差分本身不是精确真值。

3.2 逐样本和逐时间核验

若 g_A 是 A 内三项平均损失梯度， g_B 是 B 内两项平均损失梯度，整批 token-mean 梯度是 $(3/5)g_A + (2/5)g_B$ 。直接取 $(g_A + g_B)/2$ 对应不同目标。

暂时解绑定 W_1 、 W_2 、 W_3 ，前向数值相同但每份参数只在一个时刻使用。根据多变量链式法则，共享 W 的一个微小变化等于同时改变每个 W_t ，因此共享梯度等于各 W_t 梯度之和。这个检查可以定位“只保留最后一次使用贡献”的错误。

3.3 detach 的有限差分陷阱

完整数值函数在边界状态随参数重算时，通常仍依赖所有先前参数。detach 让自动微分把边界状态当作常数，所以它提供的是截断的代理梯度，不是这个完整数值函数的通常导数。前向相同并不强制梯度相同。

要检查一块的局部截断梯度，应先在旧参数下求边界状态并保存为常量，之后每次有限差分都使用同一个固定边界值，而非重新算边界。要检查完整 BPTT，则不能固定边界。混淆这两种函数会产生“正确实现却不通过差分”的假警报。

4 任务 D 状态边界诊断

4.1 正确实现不应改变的情况

保持 B 长度为 2，将 B3 输入改为 9、占位标签改为 1，不会改变任何有效预测、平均损失和参数梯度，B3 输入梯度仍是零。这项检查要求输入仍在声明范围内，9 是故意选择的有效大数。

交换 batch 顺序后再交换输出回来，结果应一致；分别处理各独立序列与在同一 batch 处理，也应一致。参数相同，样本间没有状态或注意力交互。未来若加入跨样本操作，这种断言需要重新审查，但本模型没有这种操作。

4.2 错误携带状态的数值证据

若 B1 错误使用 A3 的 $h = -0.083603677$ ，其 a 会变成 -0.350162206 ， $h = -0.336519389$ ，概率 $p = 0.392798587$ ；正确 B1 的 $p = 0.400371164$ 。B 预测被不相干的 A 改写；若打乱顺序，结果随之变化。这是状态泄漏，不是参数学习到的数据规律。

只在损失处 mask、不 freeze state，若读取“每条真实最后时刻”，已发生在该时刻之后的 padding 不影响这个读数；若错误读取补齐张量的统一末端，则短序列结果会改变。题目必须指定读取方式，所以不能笼统声称“只 mask loss 必然改坏所有有效预测”。状态冻结消除了这种补齐长度依赖，生产实现仍按 lengths 读取正确末端。

4.3 reset 和 detach 不能互换

reset 改变状态值，适合独立序列起点。detach 不改变值，却使损失无法跨边界追溯。保持同样参数的两次前向，detach 版本数值应逐位相同；终点损失对边界之前输入的梯度应为零。若把 detach 当 reset，实际改变了模型前向；若把 reset 当 detach，丢失了原本连续流的历史数值。

5 任务 E Jacobian 与裁剪

5.1 乘法顺序

a_t 对 $h_{前}$ 的导数是 W ； $\tanh$ 的 Jacobian 是 $\text{diag}(1-h_t^2)$ ，所以按列向量约定， $J_t = \text{diag}(1-h_t^2)W$ 。 h_4 对 h_1 的导数是 $J_4 J_3 J_2$ 。反向作用于列梯度时转置整个乘积，顺序变为 $J_2^T J_3^T J_4^T$ 。

固定标量因子的 20 次乘积为： $0.5^{20} \approx 9.536743 \times 10^{-7}$ ； $0.9^{20} \approx 0.121577$ ； $1.1^{20} \approx 6.727500$ ； $1.5^{20} \approx 3325.256730$ 。这是解释机制的数值例，不是对所有 RNN 梯度的预测。

$W=1.5$ 、 $a=0$ 时， $|J|=1.5$ 。 $a=3$ 时， $\tanh(3) \approx 0.995055$ ， $|J|=1.5(1-\tanh^2(3)) \approx 0.014799$ 。较大的 W 被饱和导数显著压小，所以仅凭 $W>1$ 不能推断实际路径会爆炸。矩阵中还要考虑方向和不同时间的非交换性。

5.2 全局裁剪与逐元素裁剪

$g=(3,4)$ 、 $\|g\|_2=5$ 、 $\tau=2$ ，全局裁剪得到 $(1.2,1.6)$ ，范数为 2，方向保持。逐元素裁剪得到 $(2,2)$ ，范数约 2.828，方向发生变化。

先裁剪各位置再求和不等同于先求和再裁剪。例如两个时间贡献 $g_1=(3,0)$ 、 $g_2=(0,4)$ ， $\tau=2$ ；分别裁剪后相加得到 $(2,2)$ ，完整梯度裁剪后是 $(1.2,1.6)$ 。因此生产实现必须先 backward 完成所有共享贡献，再全局裁剪。

PyTorch 2.7.1 的实现分母含 10^{-6} ，因此结果与理想公式存在微小区别；独立一步核验包含这个稳定项，不能误把它当 BPTT 错误。error_if_nonfinite=True 遇到非有限范数会报错，不能用裁剪把 NaN “变好”。

5.3 生产训练一步核验

手算例对所有位置损失求平均；真实训练只用末端 BCE。测试使用 `reference.full_bptt(...,last_only=True)` 构建独立末端目标：loss mask 每条只有一个 1、分母 N、状态 mask 仍覆盖真实序列。得到 NumPy 梯度后构造完整梯度范数和相同裁剪因子，再用旧参数一次性生成全部新参数。

随后调用真正的 `experiment.step`，逐参数比较更新值，再比较新参数下的全部 logit。分别核对不裁剪和很小阈值 0.001 的必然触发情形。这个测试覆盖了训练调用路径，避免“参考梯度正确，但真正训练误用了另一套归约或更新”的漏洞。

6 任务 F 固定实验的全部结果

protocol.json 在首次训练之前已固定 27 个组合。下表保留三个 delay、三个 seed、三个条件。训练结束点固定为第 320 步，无最好 seed 筛选。准确率表中 1 表示在这份有限样本上全部正确，不能解释为错误率被证明为零。

D	seed	条件	训练准确率	验证准确率	测试准确率	测试 BCE
2	7	full	1.000000	1.000000	1.000000	0.001478
2	7	full_clip	1.000000	1.000000	1.000000	0.001528
2	7	tbptt4_clip	1.000000	1.000000	1.000000	0.001528
2	19	full	1.000000	1.000000	1.000000	0.001215

D	seed	条件	训练准确率	验证准确率	测试准确率	测试 BCE
2	19	full_clip	1.000000	1.000000	1.000000	0.001251
2	19	tbptt4_clip	1.000000	1.000000	1.000000	0.001251
2	41	full	1.000000	1.000000	1.000000	0.001248
2	41	full_clip	1.000000	1.000000	1.000000	0.001272
2	41	tbptt4_clip	1.000000	1.000000	1.000000	0.001272
8	7	full	1.000000	1.000000	1.000000	0.001325
8	7	full_clip	1.000000	1.000000	1.000000	0.001340
8	7	tbptt4_clip	1.000000	1.000000	1.000000	0.008398
8	19	full	1.000000	1.000000	1.000000	0.001134
8	19	full_clip	1.000000	1.000000	1.000000	0.001161
8	19	tbptt4_clip	1.000000	1.000000	1.000000	0.003582
8	41	full	1.000000	1.000000	1.000000	0.001243
8	41	full_clip	1.000000	1.000000	1.000000	0.001260
8	41	tbptt4_clip	1.000000	1.000000	1.000000	0.000912
24	7	full	0.515625	0.500000	0.480469	0.694507
24	7	full_clip	1.000000	1.000000	1.000000	0.002698
24	7	tbptt4_clip	0.894531	0.875000	0.882812	0.312025
24	19	full	0.496094	0.500000	0.531250	0.693082
24	19	full_clip	1.000000	1.000000	1.000000	0.001116
24	19	tbptt4_clip	1.000000	1.000000	1.000000	0.001734
24	41	full	0.570312	0.460938	0.527344	0.700024
24	41	full_clip	1.000000	0.992188	0.984375	0.057050
24	41	tbptt4_clip	1.000000	1.000000	1.000000	0.014496

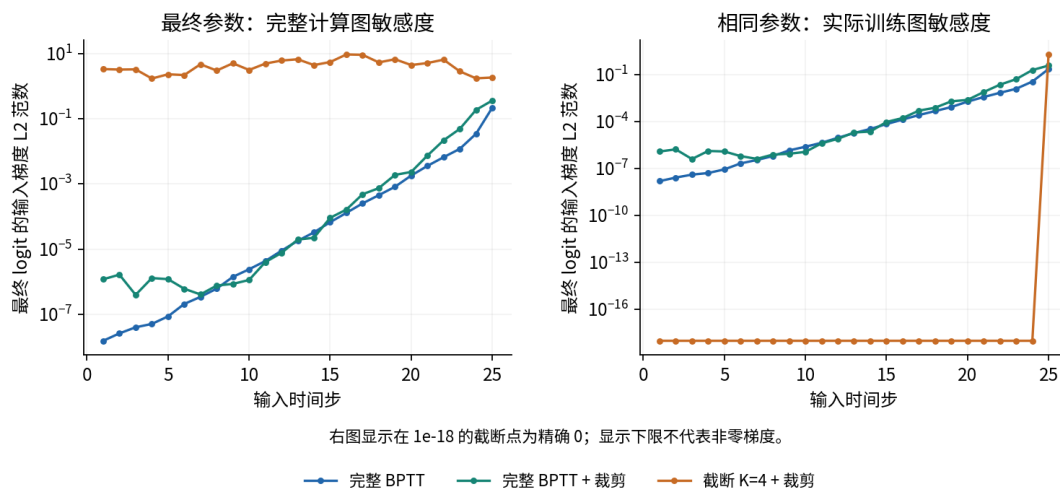
完整每一步 BCE、裁剪前后范数、每组训练参数与初始/最终输入敏感度都在 outputs 中。下表仅是公开原始记录的汇总。

延迟	条件	测试准确率均值 $\pm$ SD	测试 BCE 均值	裁剪触发均值
2	full	1.0000 $\pm$ 0.0000	0.001314	0.000%
2	full_clip	1.0000 $\pm$ 0.0000	0.001350	5.312%
2	tbptt4_clip	1.0000 $\pm$ 0.0000	0.001350	5.312%
8	full	1.0000 $\pm$ 0.0000	0.001234	0.000%
8	full_clip	1.0000 $\pm$ 0.0000	0.001254	4.688%
8	tbptt4_clip	1.0000 $\pm$ 0.0000	0.004297	5.625%
24	full	0.5130 $\pm$ 0.0283	0.695871	0.000%
24	full_clip	0.9948 $\pm$ 0.0090	0.020288	12.396%
24	tbptt4_clip	0.9609 $\pm$ 0.0677	0.109418	13.438%

6.1 截断边界控制

delay=2 时 T=3, K 的有效值取 3; 没有跨块边界, full_clip 与 tbptt4_clip 所有训练数值、参数和终点指标都应一致。这提供了一个实现等价控制。

delay=8 时 T=9, 分块为 1-4、5-8、9; delay=24 时 T=25, 分块为 1-4、5-8、9-12、13-16、17-20、21-24、25。两者最后一块都是一步。末端损失在截断图中只能对最后一步输入产生梯度; 此前输入梯度为零。它仍可通过末端使用的前一状态数值影响预测。



完整图与真实截断训练图的输入梯度不同。精确零仅为绘图方便放在显示下限，不是一个很小的非零梯度。

6.2 延迟与 seed 的真实差异

在 delay=24 下, full 的三次准确率接近 0.5、BCE 接近 $\log(2)$, 显示在这个预定预算内没有可靠学会; full_clip 三次接近或达到 1, 表现显著更好。tbptt4_clip 的 seed 7 不完全成功, 其余两个达到 1。截断组成功不能证明早期计算得到完整训练梯度, 因为直接检查已显示梯度为零。

同一 W 在末端更新后，下一次前向会在所有时刻使用更新值。早期参数无需在当前步获得跨整个序列的导数，也可能随共享更新变得能保留信号。此外，初始随机递推可能已保留一部分可读出的信息。这里只指出可能机制；要区分哪一种占主导，还需预先设计冻结 W 、冻结读出、改变干扰和分块相位的对照。

7 任务 G 有边界的结论

7.1 可直接写入实验报告的结论

在预定的延迟符号任务、 $H=12$ 、SGD 320 步及三个初始化 seed 下，短延迟三种设置均达到测试准确率 1。 $\text{delay}=24$ 的完整 BPTT 未裁剪组接近随机基线，而阈值 0.25 的完整裁剪组明显改善。 $K=4$ 固定分块删除了早期梯度，但多数长延迟运行仍能成功，说明状态数值保留与完整信用分配并非同一件事。本结果只支持该 toy 协议中的比较，不能证明裁剪普遍最优、截断等价于完整梯度或真实长文任务泛化。

7.2 六个问题的答复

1. 不能断言裁剪总是更好。需要不同学习率、阈值、初始化增益、容量、任务、数据和预算。裁剪效果与高学习率交互；在短延迟中准确率没有提高，甚至 BCE 略有变动。
2. 不能断言截断用了完整梯度。用同一最终参数、同一输入分别建立完整图和 detach 图，比较早期输入梯度；本例后者为精确零。任务可解性与某条训练梯度是否存在是两个命题。
3. Accuracy 只看分类是否跨过阈值，不看置信度和优化过程。BCE 区分正确但犹豫与正确且自信；梯度显示路径；曲线显示是否有不稳定、延迟学习和不同收敛速度。也不能凭更低 BCE 自动认定概率校准更好。
4. 3 个 seed 的 SD 不是 95% CI。样本少，分布未知；测试集被各 seed 重复使用，不增加独立样本数。这里不进行过度精确的显著性声明。
5. 不能从 detach 存在推出节省显存。教学代码保存全部状态。真正流式实现须限制存活计算图、明确窗口和更新时机，并测峰值内存与耗时；最好比较相同硬件和有效训练预算。
6. 重新运行固定协议用于数值复验，不引入新独立数据，也不应让测试指导后续调参。若根据测试修改阈值再报告最好结果，就让测试承担了开发集职责，应另设未见最终评估。

8 任务 H 事后反事实诊断

下表为额外诊断的全部运行。翻转仅改变 $x[:,0,0]$ 的符号及标签，干扰、首末标志、长度都保持不变。擦除把首位值置 0，仍用原标签。后者让本来唯一的类别信号消失，理论上对新独立样本无法系统超过随机，但有限测试集的准确率可以偏离 0.5。

D	seed	条件	翻转信号和标签准确率	擦除首位信号准确率
2	7	full	1.000000	0.492188

D	seed	条件	翻转信号和标签准确率	擦除首位信号准确率
2	7	full_clip	1.000000	0.492188
2	7	tbptt4_clip	1.000000	0.492188
2	19	full	1.000000	0.554688
2	19	full_clip	1.000000	0.550781
2	19	tbptt4_clip	1.000000	0.550781
2	41	full	1.000000	0.488281
2	41	full_clip	1.000000	0.496094
2	41	tbptt4_clip	1.000000	0.496094
8	7	full	1.000000	0.460938
8	7	full_clip	1.000000	0.464844
8	7	tbptt4_clip	1.000000	0.492188
8	19	full	1.000000	0.500000
8	19	full_clip	1.000000	0.500000
8	19	tbptt4_clip	1.000000	0.511719
8	41	full	1.000000	0.503906
8	41	full_clip	1.000000	0.496094
8	41	tbptt4_clip	1.000000	0.500000
24	7	full	0.523438	0.480469
24	7	full_clip	1.000000	0.472656
24	7	tbptt4_clip	0.875000	0.496094
24	19	full	0.468750	0.531250
24	19	full_clip	1.000000	0.500000
24	19	tbptt4_clip	1.000000	0.511719
24	41	full	0.472656	0.527344
24	41	full_clip	0.984375	0.500000
24	41	tbptt4_clip	1.000000	0.488281

这些结果支持检查模型对首位信号的依赖，而不能证明面对所有输入扰动都可靠。诊断是事后添加、未用于选择模型，应在图标题或方法段明确标注。梯度敏感度与反事实行为互补，但都不是完整因果证明。

8.1 一个可执行但尚未运行的后续计划

问题：固定 $K=4$ 下末块长度为 1，是否影响截断组学会任务的稳定性？假设：在相近总长度下，末块更长可提供更多末段信用分配，但其效果可能与状态动力学相互作用。

设计：设置 $T=24, 25, 26, 27, 28$ ；同一数据生成规则，控制每个 split 样本数、隐藏维度、初始化 seed 和学习率。每个 T 使用预先固定的独立数据 seed，条件包括 full_clip 和 tbptt4_clip。先固定 10 个模型 seed 与 320 步预算，所有终点均汇报，不选最好窗口。主要指标为测试 BCE 与准确率，机制指标为训练图中非零输入梯度覆盖的实际末块长度。另记录运行时间和峰值内存，但不宣称当前教学实现具备节省内存性质。

选择与停止：在看测试前锁定全部设置；数值失败仍保留；最多执行完整预定组合，不根据好坏追加 seed。资源预算必须先通过一个小规模的开发计时估算。最终结论分清“长度变化”与“末块相位变化”仍可能混杂；若需隔离相位，可保持同一 T 而改变首次边界位置，再另开控制实验。

此计划尚未执行，不得把预期写成已经发生的结果。改用门控结构、添加中间损失或训练更久也应作为新实验，不能偷偷替换本包原始 27 组。

9 复验范围与可恢复文件

数值检查覆盖标量和矩阵 BPTT、有限差分、共享参数汇总、生产一步、状态边界、mask、detach、裁剪和输入拒绝。正常与 -O 脚本真实执行；Notebook 从新实际 IPython 内核完整执行并保存 PNG。执行方式未测试浏览器 Jupyter 界面、外部 kernel socket、GPU、float32、极端长度或任意外部数据。

作者检查记录只说明已完成的验证，不替代独立审阅。本包的 freeze-sha256.json 固定公开产物；修改任何文件后旧冻结不再代表新内容。科学结论依赖公开数据、代码、协议与原始运行，而不是“测试项目数量很多”。

更多原始来源见 lecture.md 第 13 节与 sources.md：Pascanu 等的原始 RNN 训练困难研究，以及对应版本的 PyTorch 官方 detach、梯度裁剪文档与 RNN 源码。纸笔例、图表、实验数据及解释由本教程独立制作。